

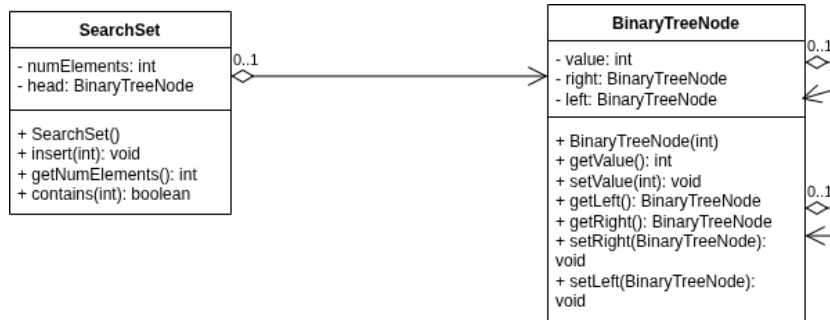
1. (b) Imperative programming languages describe a sequence of operations that change the program's state with side effects. Functional programming languages use pure functions that operate without any side effects, only returning new data.
 - (c) Primitives: d, f. References: i, l, k, t, c. Classes: Double, Computer, LinkedList
Objects: l, k.
 - (e) ii. If 2 methods exist with the same paramters and paramter types, then they could not have different return types, as when it was invoked the compiler/interpreter would not know which one to use.
 - (i) The constructor has been defined as a void type, but a constructor shouldn't have a type, so the Test() function is an unused function, and java builds an empty constructor. So when new Test() is called, only the empty constructor is called, and x is defined as 0.
2. (a) This enables rules to applied to what data can be set to an attribute.
 - (b) The advantage is that it leads to less boilerplate code needing to be written. However it relies on everyone knowing and following the convention, which can't be strictly enforced.
 - (c) i. All fields would need to be declared as final, and the class should be declared as final.
 - ii. A. As this prototype returns a void, for the mutable version, it could modify the calling Vector2D in-place. If Vector2D is immutable, since neither vector can be modified, it couldn't be implemented.
 - B. Possible for both immutable and mutable, and the code would be the same for both, as only the value of attributes need to be accessed, and can be passed into a new Vector2D.
 - C. As with the previous question, this method would would not change wether the class is mutable or immutable.
 - D. Since a static method doesn't require an instansiated object, and a new Vector can be created to store the result of the addition, it would be the same for immutable and mutable.
 - iii. Put Immutable before the name. Provide no setters.
- (b) ii. UML diagram



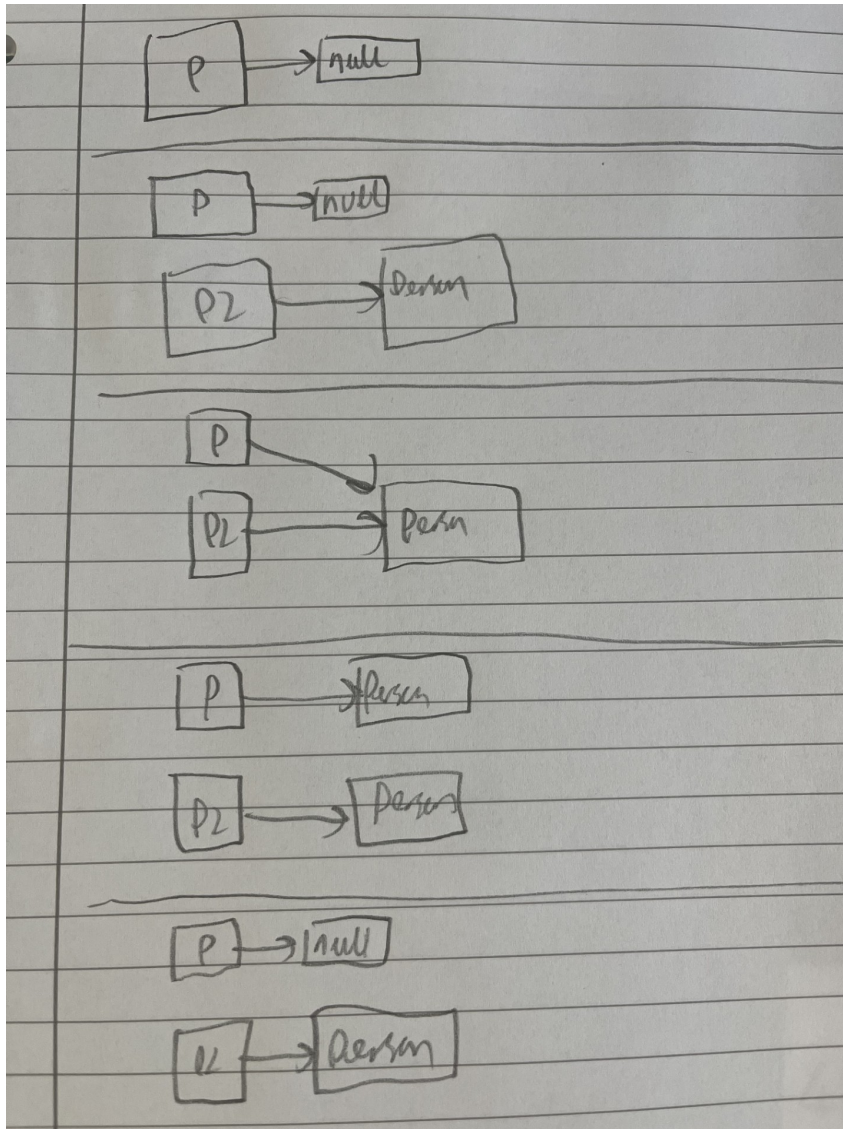
- (c) This class violates SRP as it performs multiple actions that arent directly related to one piece of data. `fetchDataFromDatabase()` performs actions related to a database, so should be in some sort of **Database** class. `parseXMLData()` performs actions related to XML files so should be in a class such as **XMLParser**.
- (d) iii. `hashCode()` has to be implemented so that when trying to insert into a data structure that uses hashing, different objects of the same class will get given different hashes. `equals()` has to be implemented so if you try to insert 2 identical objects into a data structure that uses hashing, if java can't compare them, they will both get stored in different buckets, when only one was needed.



(e) UML diagram



3. (a) References enforce that the pointer points to something that actually exists and is not null, which can prevent program crashes.
- (b) Step by step simple illustration of what happens in the memory



- (c) The output of this program will be

1 1
2 2

This is because Java is pass by value, so when `add` is called with `x` and `y` as integers, their value is copied onto the stack at a new location, and then that variable is modified, which cannot be accessed when the function returns as the stack pointer will be below where that variable was. When `add` is called on an array, its memory address is passed as the value, so even though it is stored as a new local variable inside the function, it still points to the same location in memory, and so will actually update the values in the array.

- (e) The main advantage of passing primitives by reference means that methods and functions would be able to modify primitive types, without having to define any wrapper class around them, or support pointers, which are not memory safe. However, since primitives are, on average, the same size as a memory address, it now doubles the amount of space required to store a primitive. Also, references are created in the heap and not the stack, so requiring heap space for every single primitive could easily lead to a lot of fragmentation. Furthermore, non-primitive types still can't be stored directly in memory, so their value will themselves be a reference/pointer. Passing these by reference, will now create 2 layers of references needed to access the variable, which increases code complexity, and increases space needed to store the references.
4. (a) Inheritance is an is-a relationship, i.e. if A inherits from B, then A should be able to be used in all places where B could be used. However, a 3D vector is not a 2D vector, and they cannot be used in the same places, as any operation that is defined for 2D vectors would have to be defined differently for 3D vectors, and so they cannot be used interchangeably.
- (b) The code presented in `NoAccessModifier.java` compiles, demonstrating access from a class declared outside `NoAccessModifier`, and a class that inherits from `NoAccessModifier`. This suggests that declaring a member field is automatically assigned public.
- (d) Both UML diagrams



